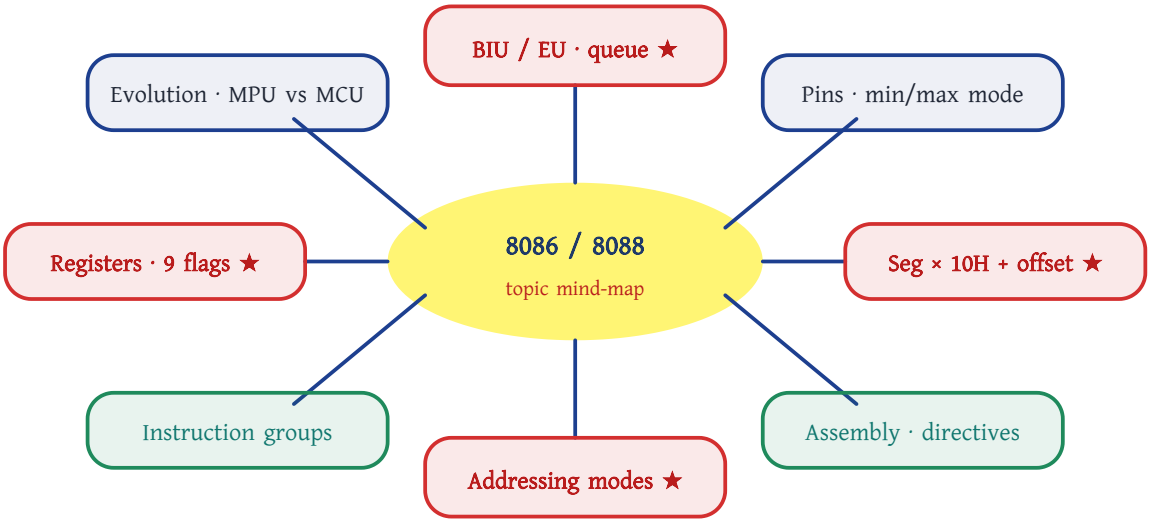
The 8086/8088 Microprocessor

BIU/EU · registers & flags · segmentation · addressing · assembly: exam-ready in 11 sheets



↑ the whole lesson on one page: red = high-yield, green = programming

Syllabus coverage

"Introduction to microprocessor: overview of computer architecture, **evolution of microprocessors**, difference between microprocessor and microcontroller; Introduction to 8086/8088: **basic architecture of 8086, memory segmentation, flags, addressing modes**, pins & signals, single and multi-processor systems; Microprocessor programming: instruction sets, introduction to assembly language programming; Tools: assemblers, debuggers, development systems". Bus controller, 8255A, timers, serial I/O, interrupts and DMA: other lessons of this chapter.

High-yield list

① 16-bit data, 20-bit address, 1 MB
② queue 6 B (8086) / 4 B (8088)
③ PA = seg × 10H + offset
④ 9 flags: 6 status + 3 control
⑤ BP → SS, BX/SI/DI → DS
⑥ reset: CS:IP = FFFF:0000

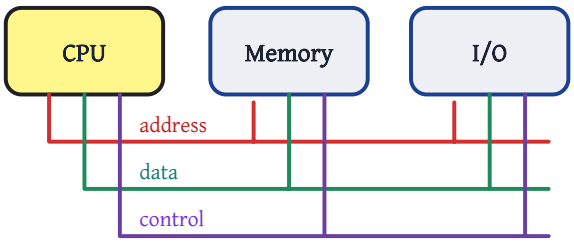
Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

- ✓ Microprocessor, buses, evolution ..... p. 2
- ✓ 8086 vs 8088, BIU/EU, queue ..... p. 3
- ✓ Registers & the 9 flags ..... p. 4
- ✓ Segmentation & physical address ... p. 5
- ✓ Banks, pins, min/max mode ..... p. 6
- ✓ Addressing modes ..... p. 7
- ✓ Instruction set ..... p. 8
- ✓ Programs, directives, tools ..... p. 9
- ✓ Night-before sheet ..... p. 10
- ✓ Practice MCQs + written ..... p. 11

1 Microprocessor & the Computer Around It

Definition: A microprocessor (MPU) is a **programmable, clock-driven, register-based IC** that holds the whole CPU (ALU + control unit + registers) on one chip. It fetches instructions from memory, decodes and executes them; memory and I/O are **outside** the chip.



the three-bus system

Bus	Direction	Its width decides
Address	**uni**	memory size = 2^n
Data	**bi**	word length (bits)
Control	mixed	RD, WR, M/IO

Instruction cycle = fetch → decode → execute. It has one or more **machine cycles**, each of several **T-states** (clock periods). n address lines → 2^n bytes: 16 → 64 KB, **20 → 1 MB**, 32 → 4 GB.

Point	Microprocessor (MPU)	Microcontroller (MCU)
On the chip	CPU only	CPU + RAM + ROM + I/O + timers
Memory · use	von Neumann · general PCs	usually Harvard · embedded, IoT
Example	8085, 8086, Core i7	8051 (4 KB ROM, 128 B RAM), ATmega328

2 Evolution of Microprocessors

Chip	Year	Data	Address → size	Remember it for
4004	**1971**	4-bit	12 → 4 KB	first commercial MPU
8008 · 8080	1972 · 74	8-bit	14 · 16 → 16 · 64 KB	8080: first popular 8-bit
8085	1976	8-bit	16 → 64 KB	single +5 V, AD0–AD7
8086 · 8088	**1978** · 79	**16-bit**	**20 → 1 MB**	8088: IBM PC (1981)
80286	1982	16-bit	24 → 16 MB	protected mode
80386 · 80486	1985 · 89	**32-bit**	32 → 4 GB	paging · FPU + cache
Pentium	1993	32-bit	32 → 4 GB	superscalar (2 pipelines)
AMD64 · Core 2	2003 · 06	64-bit	48 (virtual)	x86-64 · multi-core

Exam trap: The Pentium is a **32-bit** processor (its *external data bus* is 64 bits wide). 64-bit x86 arrived with AMD64 (Opteron, 2003). The data bus sets the "bit" size, not the address bus.

3 8086 at a Glance & 8086 vs 8088

8086 facts:

Intel **1978** · 40-pin DIP · +5 V · 29,000 transistors

16-bit ALU, registers and data bus

20-bit address bus → 2^{20} = 1 MB

I/O: 16-bit port address → 64K ports

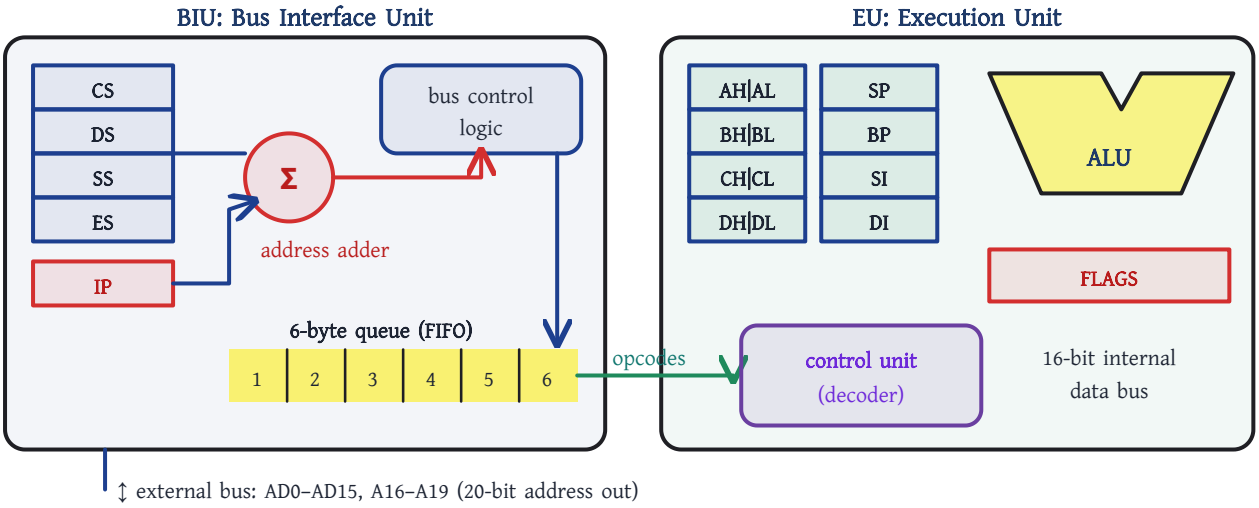
Clock 5, 8 or 10 MHz

256 interrupt types (lesson mp4)

Feature	8086	8088
External data bus	**16**	**8**
Instruction queue	**6 B**	**4 B**
Pin 34	$\overline{\text{BHE}}$/S7	$\overline{\text{SS0}}$
Memory/IO pin	M/$\overline{\text{IO}}$	IO/$\overline{\text{M}}$
Multiplexed lines	AD0–AD15	AD0–AD7
Word access	1 or 2 cycles	always 2

Same EU, same registers, same instruction set, same 1 MB: the 8088 is an 8086 with an 8-bit outside bus, chosen for the IBM PC because 8-bit support chips were cheap.

4 Two Units: BIU and EU



the BIU talks to the outside world; the EU only executes

BIU (outside work)	EU (inside work)
Fetches instruction bytes into the **queue**	Takes opcodes from the queue, executes
Makes the 20-bit physical address (Σ)	Computes the 16-bit offset (**effective address**)
Reads/writes memory & I/O operands	ALU work, sets the flags
Holds CS, DS, SS, ES, IP	Holds AX–DX, SP, BP, SI, DI, FLAGS

Memory trick: **BIU = the shopper** (goes out on the bus, fills the queue), **EU = the cook** (never leaves the kitchen, uses what is in the queue).

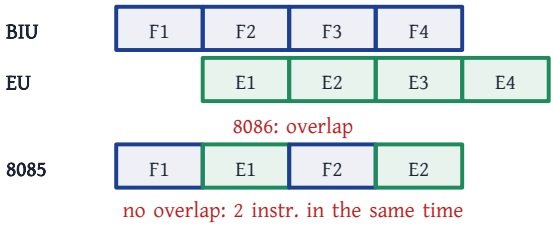
Exam trap: The EU makes the 16-bit **offset**; the 20-bit **physical address** is made in the **BIU**. IP belongs to the BIU, not the EU.

5 Queue & Pipelining

While the EU executes, the BIU **prefetches the next bytes**: fetch and execute overlap (2-stage pipeline).

8086 refills at **≥ 2** free bytes; 8088 at **≥ 1**.

On **JMP / CALL / RET / INT** the queue is **flushed** and refilled from the new CS:IP.



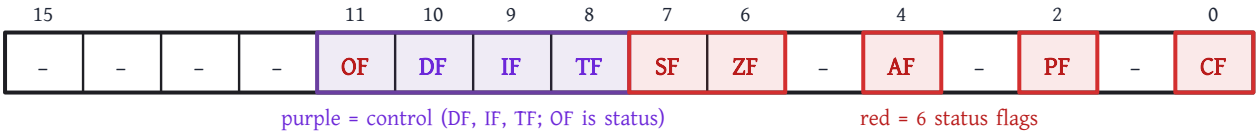
6 Register Set: 14 Registers, All 16-bit

Group	Registers and special job
General (data)	**AX** accumulator (MUL, DIV, IN, OUT) · **BX** base (only general reg. usable as a pointer) · **CX** count (LOOP, REP, shift count in CL) · **DX** data (port address, high word of MUL/DIV). Each splits into 8-bit halves: AH/AL …
Segment	**CS** code · **DS** data · **SS** stack · **ES** extra (string destination)
Pointer & index	**SP** stack top · **BP** base pointer (stack frame) · **SI** source index · **DI** destination index
Others	**IP** offset of the next instruction (not directly writable) · **FLAGS**

Default pairs: **CS:IP** · SS:SP · SS:BP · DS:BX · DS:SI · ES:DI (string destination).

> 4 + 4 + 4 + 2 = 14 registers

7 Flag Register: 9 of 16 Bits Used



Flag	Bit	= 1 when …	Flag	Bit	= 1 when / means …
CF	0	carry out of / borrow into MSB	OF	11	signed result out of range
PF	2	low byte has **even** no. of 1s	TF	8	single-step after each instr.
AF	4	carry bit 3 → 4 (BCD)	IF	9	INTR enabled (STI / CLI)
ZF	6	result = 0	DF	10	strings **decrement** (STD)
SF	7	MSB of result = 1	6 status + **3 control** = 9 flags		

> **Memory trick:** Status flags, bit 0 upward: "**C**an **P**eople **A**lways **Z**ip **S**afely? **O**K": CF 0, PF 2, AF 4, ZF 6, SF 7 … OF 11 (bits 0-2-4-6-7, then 11). Control = "**T-I-D**": TF 8, IF 9, DF 10.

Exam trap: Only AX, BX, CX, DX split into 8-bit halves; SP, BP, SI, DI do not (no "SIL" on the 8086). There is no instruction that loads IP directly: JMP, CALL, RET and INT change it.

8 Flags in Action: Worked Examples

Instructions	Result	CF	PF	AF	ZF	SF	OF
MOV AL,7FH · ADD AL,01H	80H	0	0	1	0	1	1
MOV AL,0FFH · ADD AL,01H	00H	1	1	1	1	0	0
MOV AL,80H · SUB AL,01H	7FH	0	0	1	0	0	1

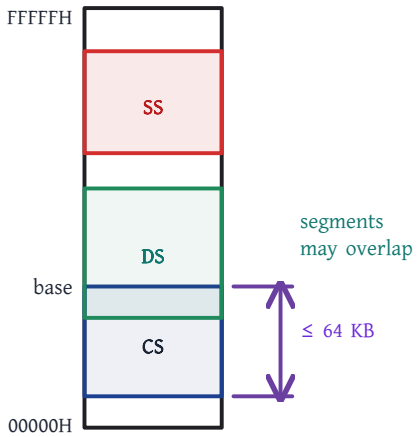
Exam trap: CF is for **unsigned** overflow, OF for **signed**. +127 + 1 → −128 sets OF but not CF; FFH + 1 sets CF but not OF (−1 + 1 = 0 is correct). PF looks at the **low 8 bits only**.

9 Memory Segmentation

Why: Registers are 16-bit (max 64 KB) but the address bus is 20-bit (1 MB). The 1 MB is viewed as **segments of up to 64 KB**, each starting on a 16-byte (**paragraph**) boundary. A **logical address** = segment : offset.

Formula: **PA = Segment × 10H + Offset**
× 10H = shift left 4 bits = append one hex 0

1. CS:IP = 1234:5678 → 12340 + 5678 = **179B8H**
2. CS:IP = 2000:0150 → 20000 + 0150 = **20150H**
3. DS = 3000H → spans **30000H – 3FFFFH**
4. PA 4A3B2H, DS 4000H → offset **A3B2H**



1 MB as 64 KB windows

Many names, one byte ✍️:
1000:2345 → 10000 + 2345 = 12345H
1234:0005 → 12340 + 0005 = 12345H
1200:0345 → 12000 + 0345 = **12345H**

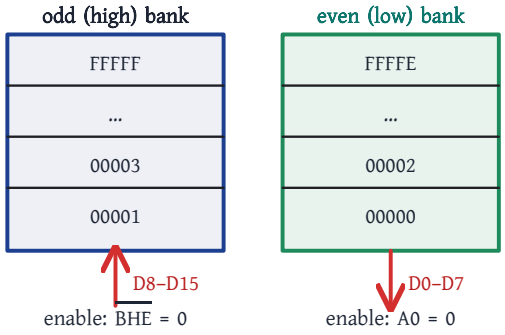
Stack & byte order ✍️:
SS = 5000H, SP = 1000H, AX = 1234H
PUSH AX → SP = **0FFEH** (SP − 2 first)
[50FFEH] = 34H, [50FFFH] = 12H
little-endian: low byte at the lower address

Memory trick: "**Add a zero, then add**": write the segment with a 0 on its right, line up the offset under it, add in hex.

Exam trap: FFFF:0010 → 100000H needs 21 bits; the 8086 **wraps round to 00000H**. The highest FFFF:FFFF = 10FFEFH → 0FFEFH.

Advantages: relocatable programs (change only the segment register), separate code/data/stack, 16-bit pointers within a segment. **Limits:** 64 KB per segment; 16 non-overlapping segments fill 1 MB.

10 Memory Banks: $\overline{BHE}$ and A0

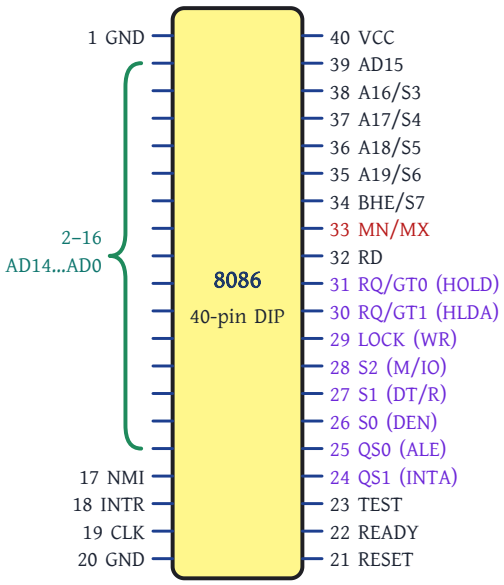


$\overline{BHE}$	A0	Transfer
0	0	**whole word** (even address)
0	1	odd byte only (D8–D15)
1	0	even byte only (D0–D7)
1	1	none (idle)

Each bank = **512 KB**, addressed by A1–A19.

Exam trap: A word at an **odd** address (e.g. 2001H) is unaligned: it needs **2 bus cycles** (odd byte first, then the even byte at 2002H). An even-address word needs one. Keep word data on even addresses (**EVEN** directive).

11 Pins & Signals; Minimum vs Maximum Mode



purple: max-mode name (min-mode name)

	Minimum	Maximum
MN/$\overline{MX}$	= 1 (V_{CC})	= 0 (GND)
System	one CPU	multi-CPU, 8087
Control	by the 8086	**8288** (S0–S2)
Bus grant	HOLD/HLDA	RQ/GT0, 1

- **ALE** latches the address off AD0–AD15 (8282/74LS373)
- **DT/$\overline{R}$**, **$\overline{DEN}$** steer the 8286 transceivers
- **READY** low → wait states (Tw)
- **$\overline{TEST}$** is checked by WAIT (8087 BUSY)
- **QS0, QS1** = queue status for the 8087
- S4 S3: 00 ES, 01 SS, 10 CS/none, 11 DS

Reset & timing: After RESET: **CS = FFFFH, IP = 0000H** → first fetch at **FFFF0H**; DS = SS = ES = 0, flags cleared (IF = 0), queue empty. A bus cycle = **4 T-states** (T1 address, T2–T4 data); at 5 MHz, T = 200 ns → 800 ns per cycle. Clock (8284A) and bus controller details: lesson mp2.

Multiprocessor configurations: coprocessor (8087 NDP shares the bus, watches QS0/QS1) · closely coupled (8089 I/O processor) · loosely coupled (each CPU has a local bus, **8289 bus arbiter** on the shared bus). The

LOCK prefix keeps the bus during a read-modify-write.

12 Addressing Modes

Definition: An addressing mode is the way an instruction **specifies where its operand is**: in the instruction, in a register, or in memory at an **effective address** (EA, the 16-bit offset) that the EU computes.



any part may be missing · PA = segment × 10H + EA

every memory mode is a special case of base + index + displacement

Worked set (use for every row): DS = 2000H, SS = 3000H, BX = 0100H, BP = 0200H, SI = 0050H, DI = 0030H.

Mode	Example	Operand / EA	PA
Immediate	MOV AX, 1234H	data inside the instruction	–
Register	MOV AX, BX	data in a register (fastest)	–
Direct	MOV AX, [1234H]	EA = 1234H	21234H
Register indirect	MOV AX, [BX]	EA = BX = 0100H	20100H
Based	MOV AX, [BX+10H]	EA = 0110H	20110H
Indexed	MOV AX, [SI+10H]	EA = 0060H	20060H
Based-indexed	MOV AX, [BX+SI]	EA = 0150H	20150H
Based-indexed + disp	MOV AX, [BX+SI+10H]	EA = 0160H	20160H
BP-based (stack)	MOV AX, [BP+DI+10H]	EA = 0240H, segment **SS**	**30240H**

Other modes	Example	Note
String	MOVSB	source DS:SI, destination ES:DI, implied
I/O port: direct / indirect	IN AL, 80H · IN AL, DX	8-bit port number (0–FFH) · DX for 0–FFFFH
Relative (branch)	JNZ NEXT	IP + signed displacement
Implied	CLC, STD, CBW	operand is fixed by the opcode

Memory trick: "**BP lives on the Stack**": any address built with BP (or SP) uses SS; BX, SI, DI use DS. Override with a prefix: MOV AX, ES:[BX].

Exam trap: Only **BX, BP, SI, DI** can sit inside [ ]: MOV AX,[CX] and [DX] are illegal, as are [BX+BP] and [SI+DI] (two bases / two indexes).

13 Instruction Groups

Group	Instructions	Point to remember
Data transfer	MOV, XCHG, PUSH, POP, IN, OUT, LEA, LDS, LES, XLAT, LAHF, SAHF, PUSHF, POPF	MOV / PUSH / POP change **no flags**
Arithmetic	ADD, ADC, SUB, SBB, INC, DEC, NEG, CMP, MUL, IMUL, DIV, IDIV, CBW, CWD, DAA, DAS, AAA, AAS, AAM, AAD	INC / DEC do not touch CF
Logical, shift, rotate	AND, OR, XOR, NOT, TEST, SHL/SAL, SHR, SAR, ROL, ROR, RCL, RCR	count = 1 or CL on the 8086
String	MOVS, CMPS, SCAS, LODS, STOS + REP, REPE/REPZ, REPNE/REPNZ	CX = count, DF picks direction
Program transfer	JMP, Jcc, LOOP, LOOPE, LOOPNE, JCXZ, CALL, RET, INT, INTO, IRET	Jcc range: -128 ... +127 bytes
Processor control	CLC, STC, CMC, CLD, STD, CLI, STI, HLT, NOP, WAIT, LOCK, ESC	NOP = XCHG AX,AX (90H)

MUL / DIV operands:

MUL BL: **AX = AL × BL**

MUL BX: **DX:AX = AX × BX**

DIV BL: AL = AX ÷ BL, AH = remainder

DIV BX: AX = DX:AX ÷ BX, DX = rem.

quotient too big or ÷ 0 → type 0 interrupt

Worked ✍️:

AX = 1234H, BX = 0100H, MUL BX

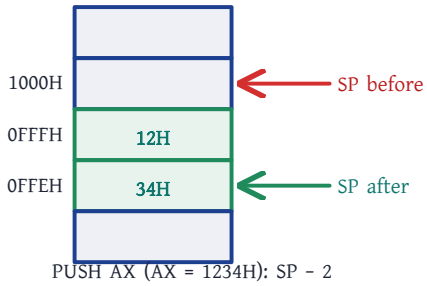
→ DX = **0012H**, AX = **3400H**

AX = 0065H (101), BL = 0AH, DIV BL

→ AL = **0AH** (10), AH = **01H** (1)

CMP AX, BX = SUB without saving

Test	Unsigned	Signed
greater	JA (CF = 0, ZF = 0)	JG (ZF = 0, SF = OF)
less	JB = JC (CF = 1)	JL (SF ≠ OF)
equal	JE = JZ (ZF = 1) · JNE = JNZ	



Memory trick: "**A**bove/**B**elow for **B**ytes without sign; **G**reater/**L**ess for **L**ife with sign". Stack: **LIFO**, grows **down**, PUSH = SP - 2 then store.

Exam trap: PUSH/POP are **16-bit only** (no PUSH AL). MOV DS, 1234H is illegal: load a segment register through a general register. MOV [BX], [SI] is illegal (no memory-to-memory MOV; use MOVS).

String set-up: source **DS:SI**, destination **ES:DI**, count CX; CLD → DF = 0 → SI, DI increase (by 1 for bytes, 2 for words); REP repeats while CX ≠ 0. LOOP = CX - 1, jump if CX ≠ 0 (flags unchanged).

14 Assembly Programs (MASM / TASM)

P1: full program, add two words

```
.MODEL SMALL
.STACK 100H
.DATA
N1   DW 9ABCH
N2   DW 8765H
SUM  DW ?
CY   DB 0
.CODE
START: MOV AX, @DATA ; DS cannot
     MOV DS, AX      ; take an imm.
     MOV AX, N1
     ADD AX, N2      ; 2221H, CF = 1
     MOV SUM, AX
     ADC CY, 0       ; CY = 1
     MOV AX, 4C00H   ; exit to DOS
     INT 21H
     END START
```

sum = **12221H** → SUM 2221H, CY 1

P2: largest of 5 bytes

```
LIST DB 25H,7AH,13H,0C4H,56H
BIG  DB ?
     LEA SI, LIST
     MOV CX, 4       ; 4 compares
     MOV AL, [SI]    ; max = 25H
NXT: INC SI
     CMP AL, [SI]
     JAE SKIP        ; unsigned
     MOV AL, [SI]    ; new max
SKIP: LOOP NXT
     MOV BIG, AL     ; C4H
```

AL: 25 → 7A → 7A → **C4** → C4 (JGE: 7AH)

P3: count the 1s in a byte

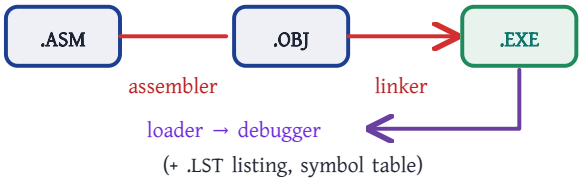
```
     MOV AL, 0B5H    ; 1011 0101
     MOV CX, 8
     XOR BL, BL      ; count = 0
AGN: SHL AL, 1       ; MSB → CF
     JNC ZRO
     INC BL
ZRO: LOOP AGN        ; BL = 5
```

P4: block copy ✍️: CLD · LEA SI,SRC · LEA DI,DST · MOV CX,10 · **REP MOVSB** → 10 bytes copied, CX = 0.

15 Assembler Directives & Tools

Directives (pseudo-ops) guide the assembler and produce **no machine code**; instructions do.

Directive	Meaning	Directive	Meaning
DB · DW · DD · DQ · DT	**1 · 2 · 4 · 8 · 10** bytes	ASSUME	seg. reg. ↔ segment
SEGMENT / ENDS	open / close a segment	PROC / ENDP	procedure
EQU · ORG · DUP	const · origin · repeat	OFFSET · PTR	offset · type
.MODEL · .DATA · .CODE	simplified segments	END label	end + entry



Memory trick: DB-DW-DD-DQ-DT = **B**yte, **W**ord, **D**ouble, **Q**uad, **T**en: 1, 2, 4, 8, 10.

Tools: assemblers MASM, TASM, NASM (two-pass: pass 1 builds the **symbol table**, pass 2 emits code) · linker LINK/TLINK · debuggers DEBUG (R registers, D dump, E enter, A assemble, U unassemble, T trace, G go, Q quit), Turbo Debugger · development systems: emulator (emu8086), trainer kit, in-circuit emulator (ICE). A **.COM** file is one 64 KB segment starting at **ORG 100H**.

Night-Before-the-Exam Sheet

Fact sheet:

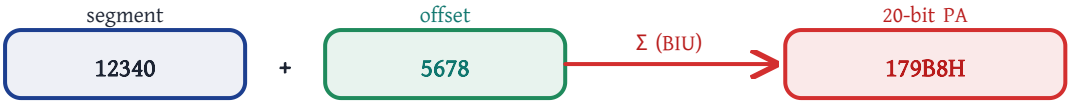
8086: 1978, 16-bit, 40 pins, +5 V	20 address lines → 2^{20} = 1 MB
Queue: 8086 = 6 B · 8088 = 4 B	BIU: CS DS SS ES IP, Σ, queue
EU: AX–DX, SP BP SI DI, ALU, flags	14 registers, all 16-bit
PA = segment × 10H + offset	segment ≤ 64 KB, on 16-byte boundary
9 flags = 6 status + 3 control	CF0 PF2 AF4 ZF6 SF7 TF8 IF9 DF10 OF11
BP, SP → SS · BX, SI, DI → DS	strings: DS:SI → ES:DI, CX, DF
$\overline{\text{BHE}}$ A0 = 00 word · 01 odd · 10 even	MN/$\overline{\text{MX}}$ = 1 min · 0 max (8288)
Reset → CS:IP = FFFF:0000 = FFFF0H	bus cycle = 4 T-states
MUL BL → AX · MUL BX → DX:AX	DB 1 · DW 2 · DD 4 · DQ 8 · DT 10

Top 12 one-liners for MCQs

1. First MPU: Intel 4004 (1971), 4-bit
2. 8085: 8-bit, 64 KB, single +5 V
3. 8088 has an 8-bit external data bus
4. Physical address is computed in the BIU
5. Fetch/execute overlap = pipelining
6. Queue is flushed on a jump or call
7. IP cannot be loaded by MOV
8. PF = 1 for even 1s in the low byte
9. DF = 1 → strings auto-decrement
10. TF = 1 → single-step mode
11. Odd-address word = 2 bus cycles
12. Directives produce no machine code

Top traps: Pentium is 32-bit · CF = unsigned, OF = signed · INC/DEC leave CF alone · [BP...] uses SS, not DS · no [CX], [DX], [BX+BP], [SI+DI] · no PUSH AL, no MOV DS, imm, no memory-to-memory MOV · FFFF:0010 wraps to 00000H · JA/JB unsigned, JG/JL signed

All tricks together: "Add a zero, then add" for PA · "C-P-A-Z-S ... O" on bits 0-2-4-6-7-11, then "T-I-D" for control flags · "BP lives on the Stack" · Above/Below = unsigned, Greater/Less = signed · stack grows down, PUSH = SP − 2 first · BIU shops, EU cooks



append one hex 0 to the segment, then add the offset

the calculation you will meet most often

✓ Practice MCQs (exam pattern, 10 marks)

1. The address bus of the 8086 is:
(a) 16 bits (b) 24 bits
(c) 20 bits (d) 32 bits

2. The instruction queue of the 8088 is:
(a) 4 bytes (b) 6 bytes
(c) 8 bytes (d) 2 bytes

3. CS = 2000H, IP = 0150H. Physical address:
(a) 02150H (b) 21500H
(c) 2150H (d) 20150H

4. MOV AX,[BP+4] takes its segment from:
(a) DS (b) SS
(c) ES (d) CS

5. The 20-bit physical address is formed by the:
(a) EU (b) ALU
(c) BIU (d) decoder

6. MOV AL,7FH then ADD AL,01H gives:
(a) OF=0, CF=1 (b) OF=1, CF=1
(c) OF=0, SF=1 (d) OF=1, SF=1

7. How many flags does the 8086 use?
(a) 9 (b) 6
(c) 16 (d) 8

8. After RESET the 8086 first fetches from:
(a) 00000H (b) FFFF0H
(c) FFFFFH (d) 00400H

9. Maximum mode needs which bus controller?
(a) 8284 (b) 8255
(c) 8259 (d) 8288

10. DW reserves how many bytes per item?
(a) 1 (b) 2
(c) 4 (d) 8

Answer key → 1-c · 2-a · 3-d · 4-b · 5-c · 6-d · 7-a · 8-b · 9-d · 10-b

Written question: Write an 8086 program that adds ten unsigned bytes stored from offset 0200H of the data segment and stores the 16-bit sum at offset 0300H (DS already set). Explain the addressing modes used. 8 marks

```
      MOV SI, 0200H   ; pointer
      MOV CX, 10      ; counter
      XOR AX, AX      ; sum = 0
NXT:  ADD AL, [SI]    ; add byte
      ADC AH, 0       ; keep carry
      INC SI
      LOOP NXT
      MOV [0300H], AX
      HLT
```

1. Set-up: SI pointer, CX count, AX = 0 — 2
2. Loop: ADD AL,[SI] (register indirect) + **ADC AH,0** turns 8-bit adds into a 16-bit sum — 3
3. INC SI, LOOP (CX – 1, repeat while ≠ 0) — 1
4. MOV [0300H],AX: direct mode; low byte at 0300H, high at 0301H — 2

Before the exam ✓

- ✓ Can draw BIU/EU and say who does what
- ✓ Can place all 9 flags on their bits
- ✓ Can compute PA, segment range, offset
- ✓ Can name the mode and PA of any MOV
- ✓ Can write a LOOP program with a trace

Also practise (written):

① min vs max mode with pins 24–31
② BHE/A0 and odd-address words
③ flags after 9AH + 7BH
④ reverse a string with SI/DI